

ToddySoft Connect S7 Web API Driver

- Ignition 8.3

Ignition 8.3 module for reaching Siemens S7-1200 and S7-1500 controllers over the **SIMATIC Web API** — the controller's own HTTPS/JSON-RPC interface.

This is a management-plane driver, not a process-data path.

The Web API reaches the diagnostic buffer, active alarms and the operating mode — surfaces the S7 driver cannot. In exchange it is roughly **an order of magnitude slower per tag** and has **no change notification on the wire**, so every subscription is polled.

Use the S7 module for cyclic process data. Pick this one for diagnostics and non-tag surfaces, or where network policy permits nothing but HTTPS. The same notice appears at the top of the device configuration form, because that is the last point at which the choice is cheap to change.

The controller also closes idle connections within seconds. That is expected and handled — the driver establishes connections per operation rather than holding one open — so a device that appears to "reconnect constantly" in the logs is working as designed.

This is a management-plane driver, not a process-data path.

The SIMATIC Web API reaches the diagnostic buffer, active alarms and the operating mode - surfaces the S7 driver cannot. It is roughly an order of magnitude slower per tag and has no change notification on the wire, so subscriptions are polled. **Use the S7 driver for cyclic process data.**

The controller also closes idle connections within seconds, which is expected and handled - connections are established per operation rather than held open.

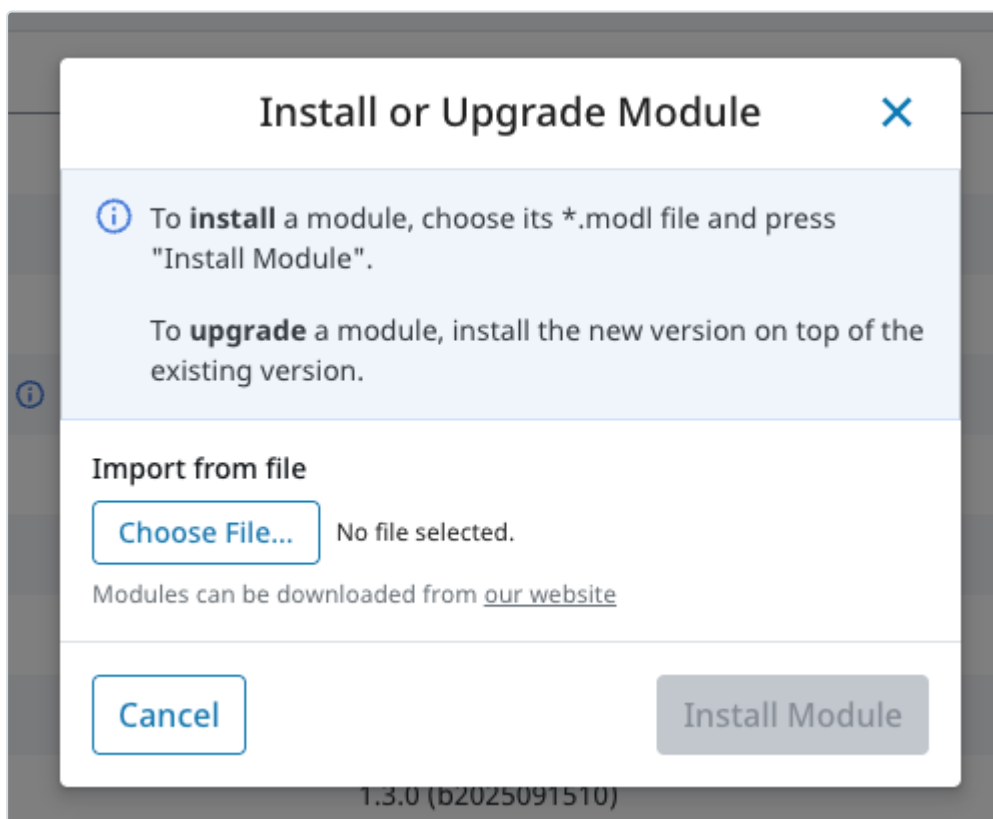
Installation

This module is for Ignition 8.3 only. It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `...-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.

Ignition 8.0 and earlier are not supported and cannot be. The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: ...opcua.server.api.AbstractDeviceModuleHook`.

Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**
3. Select the `ToddySoft-Connect-S7-Web-API-Driver-8.3.modl` file and upload it.



4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.

☐	VENDOR NAME	NAME	VERSION	LICENSE	STATUS	
☐	Inductive Automation, LLC	UDP and TCP Drivers 🔗	7.3.0 (b2025091510)	Trial	🟢 ACTIVE	⋮
☐	Inductive Automation, LLC	Omron Driver 🔗	5.3.0 (b2025091510)	Trial	🟢 ACTIVE	⋮
☐	Inductive Automation, LLC	SFC 🔗	6.3.0 (b2025091510)	Trial	🟢 ACTIVE	⋮
☐	ToddySoft GmbH	ToddySoft Connect S7 Web API Driver 🔗	1.3.0-SNAPSHOT (b0)	Trial	🟢 ACTIVE	⋮
☐	ToddySoft GmbH	ToddySoft Connect Beckhoff ADS Driver 🔗	1.2.0-SNAPSHOT (b0)	Trial	🟢 ACTIVE	⋮
☐	ToddySoft GmbH	ToddySoft Connect S7 Driver 🔗	1.3.0-SNAPSHOT (b0)	Trial	🟢 ACTIVE	⋮

Before you start: the controller side

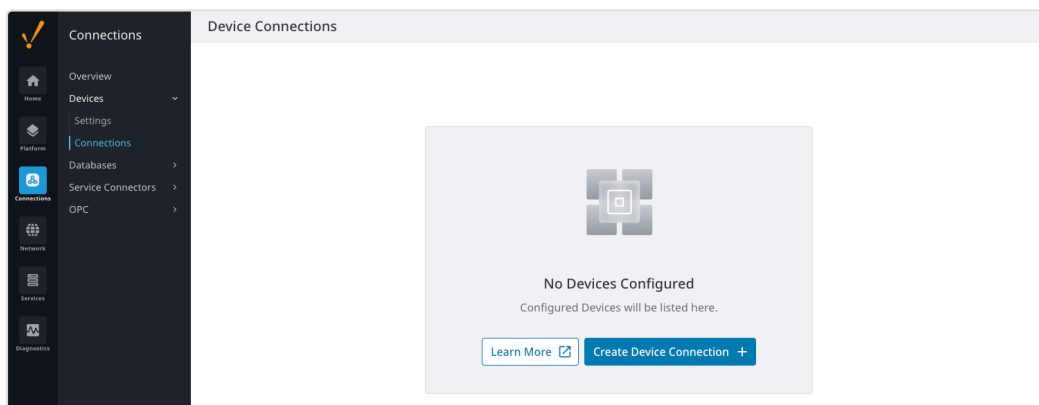
Two things must be true on the controller before a device here can connect, and neither is something this module can do for you:

- **The Web API must be enabled** for the CPU in the TIA project (*Protection & Security > Web server*), and the web server itself switched on.
- **The account must hold the Web API permission.** A user that can log into the controller's web pages is not automatically permitted to use the API; the permission is granted separately in the TIA project.

Without the second, the connection succeeds at the TLS layer and the login is then refused — which reads like a wrong password.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.



2. Click **Create new Device...** and select **ToddySoft Connect S7 Web API Driver** as the device type.

Create Device Connection

1. Select Driver

2. Configure Device

☐ TCP Driver

☐ ToddySoft Connect Beckhoff ADS Driver
 ToddySoft Connect driver for Beckhoff TwinCAT ADS/AMS protocol

☐ ToddySoft Connect CtrlX Driver
 ToddySoft Connect driver for Bosch Rexroth CtrlX protocol

☐ ToddySoft Connect EIP Driver
 ToddySoft Connect driver for EtherNet/IP (Rockwell) protocol

☐ ToddySoft Connect Modbus Driver
 ToddySoft Ignition driver for Modbus protocol

☐ ToddySoft Connect S7 Driver
 ToddySoft Connect driver for Siemens S7 protocol

☒ ToddySoft Connect S7 Web API Driver
 ToddySoft Connect driver for the Siemens SIMATIC Web API (HTTPS) on S7-1200 and S7-1500. A management-plane path for diagnostics, alarms and operating mode - roughly an order of magnitude slower per tag than the S7 driver, with no change notification on the wire. Use the S7 driver for process data.

☐ ToddySoft Connect UMAS Driver
 ToddySoft Connect driver for Schneider Electric UMAS protocol

☐ UDP Driver

Cancel

Next

3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

4. The new device then appears in the Device Connections list.

Device Connections

Connections

View 20

Create Device Connection +

☐	NAME	DRIVER	ENABLED	STATUS	
☐	Sorting-Lane-WebAPI	ToddySoft Connect S7 Web API Driver	true	CONNECTED	

Showing 1 of 1 items

Finding devices on the network

When you create a device, the configuration form carries a **Discover Devices** button directly beneath the **General** section, above the first settings card.

GENERAL

*Required

Name*

A unique name for this device connection.

Description

An optional description for this device connection.

Enabled

☒ Set whether this device connection is enabled or disabled within the Gateway.

Discover Devices

Discover Devices

Sweeps the gateway's own networks for controllers, then asks each one that answers whether it is actually serving the Web API before reporting it. else is not listed, and one that does not answer the probe in time is not listed either — so the result is the set of controllers you can genuinely connect this driver to, rather than everything with an open HTTPS port.

Discover Devices

Devices that answered a broadcast on this gateway's networks.

Timeout (ms)

25000

Rescan

How long to wait for devices to answer. The gateway clamps this to its own bounds.

S7-1500 - plcxb1511-c1-pn.profinet-schnittstellexb17c4a 192.168.24.66

tls

S7-1200 G2 - g2xbplcxb19157 192.168.24.52

tls

S7-1200 - plcxbsortingxblane3a48 192.168.23.30

tls

Cancel

Apply

A controller that answers on 443 with something

Selecting a result fills in **Host** and **Port**. Two things it deliberately does not do:

- **It fills in only what the device actually reported.** A field the sweep said nothing about is left alone rather than guessed at.
- **It never fills the username or password.** A broadcast tells you where a controller is, never which account may use its API — and the Web API permission is granted per account in the TIA project, so there is nothing sensible to guess.

Timeout (ms) bounds how long the gateway listens; it defaults to **25000**. The sweep itself takes roughly fifteen seconds, and the per-controller probe allows ten before giving up on a slow TLS handshake. A shorter bound stops the sweep before the answers arrive, and the panel then reports *No devices answered* over a network that is full of controllers.

If applying a result would overwrite something you have already typed, it lists what would change and asks first. Running a sweep needs permission to create device connections; being logged in is not enough.

Discovery does not work when Ignition runs inside Docker. The sweep is a broadcast, and a container sits on the Docker-internal network rather than on the plant network the controllers are on. Enter the host by hand; everything else on this page works normally.

This driver offers no *Test Connection* button. That action appears only for drivers that publish a device-setup capability the gateway can probe, and this one publishes none. The button is absent rather than present and inert. The Beckhoff ADS module does offer it.

Configuration

The configuration form is organized into the following sections, top to bottom:

General, **Connection**, **Web API Session**, **Certificate Trust**, **Timing**, **Limits** and **Audit Log**.

General

Assigns a name (and optional description) to the connection and controls whether it is enabled.

GENERAL

*Required

Name*

A unique name for this device connection.

Description

An optional description for this device connection.

Enabled

☒ Set whether this device connection is enabled or disabled within the Gateway.

Discover Devices

Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	true

If `Enabled` is not checked, no connection is established until it is enabled.

Connection

Setting	Description	Default
Host	Host name or IP address of the controller's web server	localhost
Port	HTTPS port of the controller's web server	443

The Web API is HTTPS-only. There is no plaintext mode to fall back to and no transport to select; a plain-HTTP connection is refused. TLS is the driver's only transport, which is why the settings for it appear under *Certificate Trust* rather than behind a transport selector.

CONNECTION

Host

192.168.23.30

Host name or IP address of the controller's web server.

Port

443

HTTPS port of the controller's web server. Defaults to 443.

Web API Session

Setting	Description	Default
Username	Controller account used for the Web API session	
Password	Password for the Web API account. Redacted from all log output	<i>None</i>
API Path	Path of the JSON-RPC endpoint on the controller	<code>/api/jsonrpc</code>
Browse Queries	Symbol browse query filter. Separate multiple queries with <code>;</code>	<code>**</code>

There is no anonymous access — the API authenticates every request, so a username is required. The account must hold the **Web API permission** granted in the TIA project; holding web-server access alone is not enough.

WEB API SESSION

Username

Controller account used for the Web API session. The account must hold the Web API permission, which is granted in the TIA project.

Password
☐ None
☒ Embedded
Secret SHOW

☐ Referenced

API Path

Path of the JSON-RPC endpoint on the controller. Override only for a firmware that relocates it.

Browse Queries

Query string for symbol browsing (default: "**"). Separate multiple queries with ";"

Override **API Path** only for a firmware that relocates the endpoint. The default is correct for every current S7-1200 / S7-1500 firmware.

Since ToddySoft Connect 1.2.0 the credentials are handed to the driver as an object rather than written into the connection string. They no longer reach the gateway log, a support bundle, or any string quoting the connection.

Certificate Trust

Setting	Description	Default
Verify Certificate	Validate the controller's TLS certificate	<code>true</code>
Pinned Fingerprints	Comma-separated SHA-256 fingerprints of the certificates to accept	<i>(empty)</i>
Trust Store Path	Truststore holding the CA that signed the controller certificate	<i>(empty)</i>
Trust Store Password	Password for the truststore. Leave as <i>None</i> for an unprotected store	<i>None</i>
Trust Store Type	Truststore format: <code>PKCS12</code> or <code>JKS</code>	<code>PKCS12</code>
TLS Version	Pin a protocol version, e.g. <code>TLSv1.3</code> . Leave empty to negotiate	<i>(empty)</i>

A controller out of the box serves a **self-signed certificate**, so this decision cannot be avoided. The driver's order of preference is:

CERTIFICATE TRUST

Pinned Fingerprints

Comma-separated SHA-256 fingerprints of the certificates to accept. The recommended option for a controller serving its factory self-signed certificate: secure, and it works without turning verification off.

Trust Store Path

Truststore holding the CA that signed the controller certificate. For a certificate signed by your own CA.

TLS Version

Leave empty to negotiate - empty means TLS 1.3 preferred with a TLS 1.2 fallback, which older controllers need. Pinning a version here removes that fallback.

Verify Certificate
☒
Validate the controller's certificate. Leave this on.

1. **Pinned fingerprints** — the right answer for a factory certificate. It is secure, and it works without turning verification off. When a verification fails, the error message contains the peer's actual fingerprint, ready to paste in.
2. **Your own truststore** — for a certificate signed by your own CA.
3. **The platform trust anchors** — for a certificate from a public CA.
4. **Verify Certificate off** — encryption without authentication. Development only.

Leave TLS Version empty unless you have a reason. Empty means TLS 1.3 preferred with a TLS 1.2 fallback, which older controllers need; pinning a version here removes that fallback.

Timing

Setting	Description	Default
Poll Interval (ms)	Interval between poll cycles for subscriptions	1000
Poll Interval Floor (ms)	Fastest permitted poll interval	500
Connect Timeout (ms)	Milliseconds allowed for the TLS handshake plus session login	5000
Request Timeout (ms)	Milliseconds allowed for a single JSON-RPC exchange	10000
Connection Request Budget	Requests to send on one TCP connection before it is transparently replaced	80

Every subscription is polled. The Web API has no change notification, so *Poll Interval* is the real data rate for this device — not a fallback. Setting it below *Poll Interval Floor* does not fail: the driver clamps the request to the floor and reports the clamp, rather than applying it silently. The form warns you when the interval you have typed is below the floor.

TIMING

Poll Interval (ms)

Interval between poll cycles for subscriptions. The Web API has no change notification, so every subscription is polled.

Poll Interval Floor (ms)

Fastest permitted poll interval. A faster request is clamped to this, and the clamp is reported rather than applied silently.

Connect Timeout (ms)

Milliseconds allowed for the TLS handshake plus session login.

Request Timeout (ms)

Milliseconds allowed for a single JSON-RPC exchange. Higher than the S7 driver's equivalent because HTTPS plus JSON is slower than binary framing.

Connection Request Budget

Requests to send on one TCP connection before it is transparently replaced. Some controllers answer exactly 101 requests and then close, announcing nothing. 0 disables replacement.

Request Timeout is deliberately higher than the S7 driver's equivalent, because HTTPS plus JSON is slower than binary framing. A JSON-RPC exchange that takes several seconds is not a fault.

Connection Request Budget exists for a real controller behaviour: some firmware answers exactly 101 requests on a connection and then closes it, announcing nothing. Replacing the connection at 80 requests keeps that from ever surfacing as a failed read. Set it to 0 to disable replacement — only worth doing against a controller measured not to need it.

Limits

Setting	Description	Default
List Default Count	Entries returned by a list-shaped tag such as <code>\$diag:buffer</code> when the address names no count	50
List Max Count	Upper bound on a requested count. A larger request is clamped and the result is reported as truncated	1000
Browse Array Element Limit	Largest array whose elements browsing enumerates as children. 0 means no limit	1000

An array larger than **Browse Array Element Limit** reports its bounds instead of children — never a truncated prefix, which would render as an array that is simply smaller than it is. Its elements stay addressable by hand, but the array is **not readable as a whole**: the reader decomposes an aggregate exactly as the browse tree does, so an array the tree will not enumerate is one the reader will not either.

LIMITS

List Default Count

50

Entries returned by a list-shaped tag such as `$diag:buffer` when the address names no count.

List Max Count

1000

Upper bound on a requested count. A larger request is clamped and the result is reported as truncated.

Browse Array Element Limit

1000

Largest array whose elements browsing enumerates as children. A larger array reports its bounds instead; its elements stay addressable by hand. 0 means no limit.

Audit Log

The `Audit Log` writes important diagnostic information to a dedicated log file. This file can grow quickly, so it is advised to only enable it when investigating issues.

AUDIT LOG

Audit-Log File Path

Path of the audit log file. Leave empty to disable audit logging.

Setting	Description	Default
Audit-Log File Path	Absolute path to the audit log file where events will be recorded. Leave empty to disable.	<i>(empty)</i>

NOTE: Please ensure the user the `Ignition` process is running under has write access to this path.

Tag Addressing

Symbolic only — there is no absolute-address form. The grammar follows TIA's display form:

```
"<datablock>".<member>[.<member>...][:<TYPE>]
"<plc-tag>"[:<TYPE>]
$<namespace>:<verb>[:<TYPE>][(<count>)]
```

Address	Meaning
"TestData".g_s16	An <code>INT</code> member of data block <code>TestData</code>
"TestData".g_plant.meta.str	A nested struct member
"OPC-UA-DB"."OPC Data".g_b1	A member whose name is not a plain identifier — it must be quoted
"ManualTrigger"	A PLC tag in the global tag table
\$diag:buffer[0..19]	The 20 newest diagnostic-buffer entries

Addresses are *not* interchangeable with the S7 module. That driver accepts `%DB42:0.0:B00L` absolute forms as well as symbolic ones; this one is symbolic-only and uses the quoted TIA display form. Tags cannot be moved between the two by changing the device.

The type suffix is optional; without one the controller's own type governs, which is the normal case. Members whose names contain anything but letters, digits and `_` — spaces and hyphens are common in real projects — must be quoted, and browsing does this for you.

Browsed addresses are usable verbatim. Browsing also reports array bounds (`ARRAY [1..8] OF B00L`), string capacity (`STRING(40)`) and write protection, all from the controller's own metadata.

Diagnostics through synthetic tags

Non-tag surfaces are reached through a reserved `$` namespace on the ordinary read/write path — the reason to choose this module in the first place:

Tag	Direction	Meaning
<code>\$diag:buffer / \$diag:buffer[0..49]</code>	read	Diagnostic buffer, newest first
<code>\$diag:alarms</code>	read	Active alarms
<code>\$control:operating-mode:STRING</code>	read / write	Read the mode, or write <code>run</code> / <code>stop</code> to request a change
<code>\$session:permissions</code>	read	The account's effective rights

A variable literally starting with `$` cannot be addressed — the namespace is reserved.

Writing `$control:operating-mode` is plant-affecting, and the acknowledgement does not mean what it looks like. An `OK` means the request was **queued**, not that the mode changed — and a second request is queued too, not rejected. Measured on an S7-1200 G2: three contradictory commands sent 8–10 ms apart (`run` , `stop` , `run`) were each acknowledged and **all three executed in order**, stopping and restarting the CPU in about 135 ms with no error reported at any point.

- **Never retry a mode change because the acknowledgement was slow.** The first one is already queued; a retry is a second execution.
- **Guard the UI.** A double-clicked STOP will be carried out twice.
- **Confirm by reading, not by the acknowledgement.** A read immediately after the write reports the *old* mode, which is indistinguishable from a refusal. Poll until it settles and treat a timeout as the failure.

Arrays and structures

Aggregates read whole; they never write whole. The wire has no aggregate transfer: reading an array by name is refused by the controller, and so is reading a structure. The driver serves reads anyway, by decomposing the aggregate into the addresses the controller *does* accept and reassembling the answer. Writes stay refused, deliberately

— a decomposed read that fails partway is a retryable bad observation, while a decomposed write that fails partway leaves the controller holding a mixture of new and old members with no rollback. **Write the members or the elements individually.**

One failed member fails the whole tag, with the message naming the member; a structure delivered with holes would look complete while quietly carrying nulls.

Three details matter if you compose subscripts by hand rather than browsing:

- **Indices are comma-separated inside one bracket group.** `g_matI16_2x3[1,1]` works; `g_matI16_2x3[1][1]` is rejected as an invalid address.
- **The declared lower bound is preserved, never normalised.** In the same block one array can start at 0 and another at 1. Assuming a base addresses the wrong element.
- **An intermediate row of a multi-dimensional array is itself an array** — readable by decomposition, not writable.

Configure **Browse Queries** to control which symbols appear when browsing tags in the Ignition Designer.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

On this driver the tag group interval interacts with *Poll Interval*. The Web API is polled, so a tag group faster than the device can serve does not produce faster data — it produces requests that queue. Size the tag groups to what the controller can answer, not to what you would like.

Troubleshooting

Device shows "Faulted":

- Verify the controller's host name / IP is reachable on the configured HTTPS port
- Check that the **Web API is enabled** for the CPU in the TIA project
- Review Gateway logs: **Status > Diagnostics > Logs**

Login fails although the password is right:

- The account most likely lacks the **Web API permission**, which is granted separately from web-server access in the TIA project. The TLS layer succeeds and the login is refused, which reads exactly like a wrong password.

TLS connection fails:

- A factory self-signed certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Fingerprints**; the error message reports the controller's actual fingerprint
- For a certificate signed by your own CA, set **Trust Store Path** and **Trust Store Type**
- For an older controller, clear **TLS Version** so the TLS 1.2 fallback is available again

The logs show the connection being opened and closed repeatedly:

- Expected. The controller closes idle connections within seconds, so the driver establishes them per operation rather than holding one open. This is not a fault and needs no configuration change.

Reads fail after roughly a hundred requests:

- Some firmware answers exactly 101 requests on a connection and then closes it, announcing nothing. **Connection Request Budget** (default 80) replaces the connection before that happens; if it has been set to 0, restore the default.

Reading an array or a structure returns an error:

- Whole-aggregate *writes* are unsupported by design — write the members or elements individually. If a *read* fails, one member failed and the message names it. An array above **Browse Array Element Limit** is not readable whole at all; its elements remain individually readable.

Collection is too slow:

- This driver is roughly an order of magnitude slower per tag than the S7 driver and has no change notification. If the cycle time matters, that is the wrong module for the job — use the **S7 module** for process data and keep this one for diagnostics.

Docker related issues

Ordinary connections work normally from a container: the Web API is an outbound HTTPS connection with no identity for this end to announce. Only **Discover Devices** is affected, because its broadcast never leaves the Docker network — enter the host by hand.

VPN related issues

A remote controller reached over a VPN needs nothing special from this driver. If the connection fails, it is ordinary reachability — confirm that the HTTPS port is routed and not blocked.

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **Added:** this manual.